

Document number:	P0327R0
Date:	2016-05-29
Project:	ISO/IEC JTC1 SC22 WG21 Programming Language C++
Audience:	Evolution Working Group / Reflection Working Group / Library Evolution Working Group
Reply-to:	Vicente J. Botet Escribá < vicente.botet@nokia.com >

Product types access

Abstract

This paper proposes a library mechanism for deconstructing types that parallels the language mechanism described in Structured binding [P0326R0](#). This proposal name them *product types* The interface includes getting the number of elements, access to the n^{th} element and the type of the n^{th} element.

The main benefits of this are cheap reflection, allow automatic serialization support, automated interfaces, etc.

The wording depends on the wording of [P0326R0](#).

In addition, some of the algorithms that work for *tuple-like* access are adapted to work with *product types*.

Table of Contents

1. [Introduction](#)
2. [Motivation](#)
3. [Proposal](#)
4. [Design Rationale](#)
5. [Wording](#)
6. [Implementability](#)
7. [Open points](#)
8. [Future work](#)
9. [Acknowledgements](#)
10. [References](#)

Introduction

Defining *tuple-like* access `tuple_size`, `tuple_element` and `get<I>/get<T>` for simple classes is --

as for comparison operators ([N4475](#)) -- tedious, repetitive, slightly error-prone, and easily automated.

[P0144R2/P0217R1/P0326R0](#) proposes the ability to bind all the members of some type, at a time via the new structured binding statement. This proposal names those types *product types*.

[P0197R0](#) proposed the generation of the *tuple-like* access function for simple structs as the [P0144R2](#) does for simple structs (case 3).

We are unable to define a *tuple-like* access interface for C-arrays, as the `get<I>(arr)` cannot be found by ADL.

This paper proposes a library interface to access the same types covered by Structured binding [P0326R0](#), *product types*. The interface includes getting the number of elements, access to the n^{th} element and the type of the n^{th} element. This interface doesn't use ADL.

The wording of Structured binding has been modified so that both structured binding and the possible product type access wording isn't repetitive.

Motivation

Status-quo

Besides `std::pair`, `std::tuple` and `std::array`, aggregates in particular are good candidates to be considered as *tuple-like* types. However defining the *tuple-like* access functions is tedious, repetitive, slightly error-prone, and easily automated.

Some libraries, in particular [Boost.Fusion](#) and [Boost.Hana](#) provide some macros to generate the needed reflection instantiations. Once this reflection is available for a type, the user can use the struct in algorithms working with heterogeneous sequences. Very often, when macros are used for something, it is hiding a language feature.

Algorithms such as `std::tuple_cat` and `std::experimental::apply` that work well with *tuple-like* types, should work also for *product* types. There are many more of them; a lot of the homogeneous container algorithm are applicable to heterogeneous containers and functions, see [Boost.Fusion](#) and [Boost.Hana](#). Some examples of such algorithms are `fold`, `accumulate`, `for_each`, `any_of`, `all_of`, `none_of`, `find`, `count`, `filter`, `transform`, `replace`, `join`, `zip`, `flatten`.

[P0144R2/P0217R1/P0326R0](#) proposes the ability to bind all the members of a *tuple-like* type at a time via the new structured binding statement. [P0197R0](#) proposes the generation of the *tuple-like* access function for simple structs as the [P0144R2](#) does for simple structs (case 3 in [P0144R2](#)).

The wording in [P0217R1/P0326R0](#), allows to do structure binding for C-arrays and allow bitfields as members in case 3 (built-in). But

- bitfields cannot be managed by the current *tuple-like* access function `get<I>(t)` without returning a

bitfields reference wrapper, so [P0197R0](#) doesn't provides a *tuple-like* access for all the types supported by [P0217R1](#).

- we are unable to find a `get<I>(arr)` overload on C-arrays using ADL.

This is unfortunately asymmetric. We want to have structure binding, pattern matching and *product types* access for the same types.

This means that the *extended tuple-like* access cannot be limited to *tuple-like* access.

Ability to work with bitfields

To provide *extended tuple-like* access for all the types covered by [P0144R2](#) which support getting the size and the n^{th} element, we would need to define some kind of predefined operators

`pt_size(T)` / `pt_get(N, pt)` that could use the new *product type* customization points. The use of operators, as opposed to pure library functions, is particularly required to support bitfield members.

The authors don't know how to define a function interface that could manage with bitfield references. See [P0326R0](#) "Ability to work with bitfields only partially" for a description of the customization issues.

Parameter packs

We shouldn't forget parameter packs, which could be seen as being similar to product types. Parameter packs already have the `sizeof...(T)` operator. Some (see e.g. [P0311R0](#) and references therein) are proposing to have a way to explicitly access the n^{th} element of a pack (a variety of possible syntaxes have been suggested). The authors believe that the same operators should apply to parameter packs and product types.

Proposal

Taking into consideration these points, this paper proposes a *product type* access library interface.

Future *Product type* operator proposal (Not yet)

We don't propose yet the *product type* operators to get the size and the n^{th} element as we don't have a good proposal for the operators's name. We prefer to wait until we have some concrete proposal for parameter packs direct access.

The *product type* access could be based on two operators: one `pt_size(T)` to get the size and the other `pt_get(N, pt)` to get the N^{th} element of a *product type* instance `pt` of type `T`. The definition of these operators would be based on the wording of structured binding [P0217R1](#).

The name of the operators `pt_size` and `pt_get` are of course subject to bike-shedding.

But what would be the result type of those operators? While we can consider `pt_size` as a function and we could say that it returns an `unsigned int`, `pt_get(N, pt)` wouldn't be a function (if we want to support

bitfields), and so `decltype(pt_get(N, pt))` wouldn't be defined if the N^{th} element is a bitfield managed on [P0144R2](#) case 3. In all the other cases we can define it depending on the const-rvalue nature of `pt`.

The following could be syntactic sugar for those operators but we don't propose them yet. We wait to see what we do with parameter packs direct access and sum types.

- `pt_size(PT) = sizeof...(PT)`
- `pt_get(N, pt) = pt.[N]`

Caveats

1. `pt_size(T)`, `pt_size(T)` and `pt_get(N, pt)` aren't functions, and so they cannot be used in any algorithm expecting a function. Generic algorithms working on *product* types should take the type as a template parameter and possibly an integral constant for the indices.
2. We need to find the name for those two operators.

Product type library proposal

An alternative is to define generic functions `std::product_type::size<PT>()` and `std::product_type::get<I>(pt)` using wording similar to that in [P0217R1](#).

The interface tries to follow in some way the guidelines presented in [N4381](#).

We have two possibilities for `std::product_type::get`: either it supports bitfield elements and we need a `std::bitfield_ref` type, or it doesn't support them.

We believe that we should provide a `bitfield_ref` class in the future, but this is out of the scope of this paper.

However, we can already define the functions that will work well with all the *product types* except for bitfields.

```
namespace std {
namespace product_type {

    template <class PT>
    struct size;

    // Wouldn't work for bitfields
    template <size_t N, class PT>
    constexpr auto get(PT&& pt)

    template <size_t N, class PT>
    struct element;

}}
```

While this could be seen as a limitation, and it would be in some cases, we can already start to define a lot of algorithms.

Users could already define their own `bitfield_ref` class and define its customization point for bitfields members if needed.

Algorithms and function adaptation

`std::tuple_cat`

Adapt the definition of `std::tuple_cat` in [tuple.creation] to take care of product type

Constructor from a product type with the same number of elements as the tuple

Similar to the constructor from `pair`.

This simplifies a lot the `std::tuple` interface (See [N4387](#)).

`std::apply`

Adapt the definition of `std::apply` in [xxx] to take care of product type

NOTE: This algorithm could be moved to a *product type* specific algorithms file.

`std::pair`

piecewise constructor

The following constructor could also be generalized to *product types*

```
template <class... Args1, class... Args2>
pair(piecewise_construct_t,
    tuple<Args1...> first_args, tuple<Args2...> second_args);
```

```
template <class PT1, class PT2>
pair(piecewise_construct_t, PT1 first_args, PT2 second_args);
```

Constructor and assignment from a product type with two elements

Similar to the `tuple` constructor from `pair`.

This simplifies a lot the `std::pair` interface (See [N4387](#)).

Design Rationale

What do we loss if we don't add this *product type* access in C++17?

We will be unable to define algorithms working on the same kind of types supported by Structured binding [P0326R0](#).

While Structured binding is a good tool for the user, it is not adapted to the library authors, as we need to know the number of elements of a product type to do Structured binding.

This means that the user would continue to write generic algorithms based on the *tuple-like* access and we cannot have a *tuple-like* access for c-arrays and for the types covered by Structured binding case 3 [P0326R0](#).

Traits versus functions

Should the *product type* `size` access be a constexpr function or a trait?

Locating the interface on a specific namespace

The name of *product type* interface, `size`, `get`, `element`, are quite common. Nesting them on a specific namespace makes the intent explicit.

We can also preface them with `product_type_`, but the role of namespaces was to be able to avoid this kind of prefixes.

Namespace versus struct

We can also place the interface nested on a struct. Using a namespace has the advantage that we can use using directives and using declarations.

Using a `struct` would make the interface closed to adding new nested functions, but it would be open by derivation.

Wording

Add the following section

Product types terms

A type `E` is a *product type* if the following terms are well defined. Let `e` be a lvalue of type `E`

product type size of E

- If `E` is an array type with element type `T`, then is equal to the number of elements of `E`.
- Else, the unqualified-id `product_type_size` is looked up in the scope of `E` by class member access lookup (3.4.5 [basic.lookup.classref]), and if that finds at least one declaration, then is `e.product_type_size()`. Otherwise, then is `product_type_size(e)`, where `product_type_size` is looked up in the associated namespaces (3.4.2 [basic.lookup.argdep]). [Note: Ordinary unqualified lookup (3.4.1 [basic.lookup.unqual]) is not performed. -- end note].
- Else, if all of `E`'s non-static data members and bit-fields shall be public direct members of `E` or of the same unambiguous public base class of `E`, `E` shall not have an anonymous union member, equal to the number of non-static data members of `E`.
- Else it is undefined.

product type ith-element of E

- If the *product type size of E* is defined and `i` is less than the *product type size of E*.
 - If `E` is an array type with element type `T`, equal to `e[i]`.
 - Else, if the expression `e.product_type_size()` is a well-formed integral constant expression, equal to the following: The unqualified-id `product_type_get` is looked up in the scope of `E` by class member access lookup (3.4.5 [basic.lookup.classref]), and if that finds at least one declaration, the value is `e.product_type_get<i-1>()`. Otherwise, the value is `product_type_get<i-1>(e)`, where `product_type_get` is looked up in the associated namespaces (3.4.2 [basic.lookup.argdep]). [Note: Ordinary unqualified lookup (3.4.1 [basic.lookup.unqual]) is not performed. -- end note].
 - Else, if all of `E`'s non-static data members and bit-fields shall be public direct members of `E` or of the same unambiguous public base class of `E`, `E` shall not have an anonymous union member, equal to `e.mi` where `i`-th non-static data member of `E` in declaration order is designated by `mi`.
 - Else it is undefined.
- Else it is undefined.

product type ith-element type of E

- If the *product type size of E* is defined and `i` is less than the *product type size of E*.
 - If `E` is an array type with element type `T`, equal to `T`.
 - Else If the expression `E::product_type_element_type<i-1>::type` is a well-formed integral constant expression, equal to `E::element_type<i-1>::type`.
 - Else, the unqualified-id `product_type_element_type` is looked up in the scope of `E` by class member access lookup (3.4.5 [basic.lookup.classref]), and if that finds at least one declaration, the type is `decay_t<decltype(e.product_type_element_type(integral_constant<size_t, i>{}))>`.

- Else, the unqualified-id `product_type_element_type` is looked up in the associated namespaces (3.4.2 [basic.lookup.argdep]). [Note: Ordinary unqualified lookup (3.4.1 [basic.lookup.unqual]) is not performed. -- end note], and if that finds at least one declaration, the type is `decay_t<decltype(product_type_element_type(integral_constant<size_t, i>{}, e))>`
- Else if the *product type i^{th} -element of e* is defined the type is `decay_t< product type  $i^{th}$ -element of  $e$  >`.
- Else, if all of `E` 's non-static data members and bit-fields shall be public direct members of `E` or of the same unambiguous public base class of `E`, `E` shall not have an anonymous union member, equal to `decay_t<decltype(e.mi)>` where `i` -th non-static data member of `E` in declaration order is designated by `mi`.
- Else it is undefined.

- Else it is undefined.

If any of the previous terms is not defined the other are not defined.

Update the Structured Binding wording to make use of the previous terms

In 7.1.6.4 [dcl.spec.auto] paragraph 8 of the Structured Binding proposal

Replace

If `E` is an array,

bit-field if that member is a bit-field.

by

If the *product type size* of `E` is defined and *product type i^{th} -element* is defined for all `i` in $0..product\ type\ size$ then

- then number of elements in the identifier-list shall be equal to *product type size* of `e`.
- each `vi` is the name of an lvalue that refers to the *product type $i-1^{th}$ -element* and whose type is *product type $i-1^{th}$ -element type*.

Add a new `<product_type>` file in 17.6.1.2 Headers [headers] Table 14

Add the following section

Product type object

Product type synopsis

```

namespace std {
namespace product_type {

    template <class PT>
    struct size;

    template <size_t N, class PT>
    constexpr auto get(PT&& pt);

    template <size_t I, class PT>
    struct element;

}}

```

Template Class `product_type::size`

```

template <class PT>
struct size : integral_constant<size_t, `see below`> {};

```

Remark: if *product type size* `PT` is defined, the value of the integral constant is *product type size* `PT` else the trait is undefined.

Note: In order to implement this trait library it would be required that the compiler provides some builtin as e.g. `__builtin_pt_size(PT)` that implements *product type size* `PT`.

Template Class `product_type::element`

```

template <class PT>
struct element {
    using type = `see below`
};

```

Remark: if *product type N^{th} -element type of PT* is defined the nested alias `type` is *product type N^{th} -element type of PT*. Else it is undefined.

Note: In order to implement this trait library it would be required that the compiler provides some builtin as e.g. `__builtin_pt_element_type(N, PT)` that implements *product type element type* `N`, `PT`.

Template Function `product_type::get`

```

template <size_t N, class PT>
constexpr auto get(PT && pt);

```

Requires: `N < size<PT>()`

Returns: the *product type `N` th-element* of `pt` .

Remark: This operation would not be defined if *product type Nth-element* of `pt` is undefined.

Note: In order to implement this function library it would be required that the compiler provides some builtin as e.g. `__builtin_pt_get(N, pt)` that implements *product type Nth-element* of `pt` .

Change 20.4.1p1 [tuple.general], Header synopsis as indicated.

Replace

```
template <class... Tuples>
constexpr tuple<CTypes...> tuple_cat(Tuples&&... tpls);
```

by

```
template <class... PTs>
constexpr tuple<CTypes...> tuple_cat(PTs&&... pts);
```

Change 20.4.2 [tuple.tuple], class template tuple synopsis, as indicated.

Replace

```

// 20.4.2.1, tuple construction
...
template <class... UTypes>
    EXPLICIT constexpr tuple(const tuple<UTypes...>&);
template <class... UTypes>
    EXPLICIT constexpr tuple(tuple<UTypes...>&&);

template <class U1, class U2>
    EXPLICIT constexpr tuple(const pair<U1, U2>&);           // only if sizeof...(Types) == 2
template <class U1, class U2>
    EXPLICIT constexpr tuple(pair<U1, U2>&&);               // only if sizeof...(Types) == 2

// 20.4.2.2, tuple assignment
...
template <class... UTypes>
    tuple& operator=(const tuple<UTypes...>&);
template <class... UTypes>
    tuple& operator=(tuple<UTypes...>&&);
template <class U1, class U2>
    tuple& operator=(const pair<U1, U2>&); // only if sizeof...(Types) == 2
template <class U1, class U2>
    tuple& operator=(pair<U1, U2>&&); // only if sizeof...(Types) == 2

// allocator-extended constructors
...
template <class Alloc, class... UTypes>
    EXPLICIT tuple(allocator_arg_t, const Alloc& a, const tuple<UTypes...>&);
template <class Alloc, class... UTypes>
    EXPLICIT tuple(allocator_arg_t, const Alloc& a, tuple<UTypes...>&&);
template <class Alloc, class U1, class U2>
    EXPLICIT tuple(allocator_arg_t, const Alloc& a, const pair<U1, U2>&);
template <class Alloc, class U1, class U2>
    EXPLICIT tuple(allocator_arg_t, const Alloc& a, pair<U1, U2>&&);

```

by

```

// 20.4.2.1, tuple construction
...
template <class PT>
    EXPLICIT constexpr tuple(PT&&);

// 20.4.2.2, tuple assignment
...
template <class PT>
    tuple& operator=(PT&& u);

// allocator-extended constructors
...
template <class Alloc, class PT>
    EXPLICIT tuple(allocator_arg_t, const Alloc& a, PT&&);

```

Constructor from a product type

Suppress in 20.4.2.1p3, Assignment

, and `Ui` be the `i`th type in a template parameter pack named `UTypes`, where indexing is zero-based

Replace 20.4.2.1p15-26, Construction by

```

template <class PT>
    EXPLICIT constexpr tuple(PT&& u);

```

Let `Ui` is `product_type::element<i, decay_t<PT>>::type`.

Effects: For all `i`, the constructor initializes the `i`th element of `*this` with `std::forward<Ui>(product_type::get<i>(u))`.

Remarks: This constructor shall not participate in overload resolution unless `PT` is a *product type* with the same number elements than this tuple and `is_constructible<Ti, Ui&&>::value` is true for all `i`. The constructor is explicit if and only if `is_convertible<Ui&&, Ti>::value` is false for at least one `i`.

Assignment from a product type

Suppress in 20.4.2.2p1, Assignment

and `Ui` be the `i`th type in a template parameter pack named `UTypes`, where indexing is zero-based

Replace 20.4.2.2p9-20, Assignment by

```
template <class PT>
    tuple& operator=(PT&& u);
```

Let `Ui` is `product_type::element<i, decay_t<PT>>::type` .

Effects: For all `i` , assigns `std::forward<Ui>(product_type::get<i>(u))` to `product_type::get<i>(*this)`

Returns: `*this`

Remarks: This function shall not participate in overload resolution unless `PT` is a *product type* with the same number elements than this tuple and `is_assignable<Ti&, const Ui&>::value` is true for all `i` .

Allocator-extended constructors from a product type

Change the signatueres

```
template <class Alloc>
    tuple(allocator_arg_t, const Alloc& a, const tuple&);
template <class Alloc>
    tuple(allocator_arg_t, const Alloc& a, tuple&&);
template <class Alloc, class... UTypes>
    EXPLICIT tuple(allocator_arg_t, const Alloc& a, const tuple<UTypes...>&);
template <class Alloc, class... UTypes>
    EXPLICIT tuple(allocator_arg_t, const Alloc& a, tuple<UTypes...>&&);
template <class Alloc, class U1, class U2>
    EXPLICIT tuple(allocator_arg_t, const Alloc& a, const pair<U1, U2>&);
template <class Alloc, class U1, class U2>
    EXPLICIT tuple(allocator_arg_t, const Alloc& a, pair<U1, U2>&&);
```

by

```
template <class Alloc, class PT>
    EXPLICIT tuple(allocator_arg_t, const Alloc& a, PT&&);
```

`std::tuple_cat`

Adapt the definition of `std::tuple_cat` in [tuple.creation] to take care of product type

Replace `Tuples` by `PTs` , `tpls` by `pts` , `tuple` by `product type` , get by `product_type::get` and `tuple_size` by `product_type::size` .

```
template <class... PTs>
constexpr tuple<CTypes...> tuple_cat(PTs&&... pts);
```

std::apply

Adapt the definition of `std::apply` in [xxx] to take care of product type

Replace `Tuple` by `PT`, `t` by `pt`, `tuple` by `product_type`, `std::get` by `product_type::get` and `std::tuple_size` by `product_type::size`.

```
template <class F, class PT>
constexpr decltype(auto) apply(F&& f, PT&& t);
```

std::pair

Change 20.3.2 [pairs.pair], class template pair synopsis, as indicated:

Replace

```
template <class... Args1, class... Args2>
pair(piecewise_construct_t,
    tuple<Args1...> first_args, tuple<Args2...> second_args);
```

by

```
template <class PT1, class PT2>
pair(piecewise_construct_t, PT1 first_args, PT2 second_args);
```

Add

```C++

template EXPLICIT constexpr pair(PT&& u); ... template tuple& operator=(PT&& u);

}```

## piecewise constructor

Replace

```
template <class... Args1, class... Args2>
pair(piecewise_construct_t,
 tuple<Args1...> first_args, tuple<Args2...> second_args);
```

by

```
template <class PT1, class PT2>
 pair(piecewise_construct_t, PT1 first_args, PT2 second_args);
```

## Constructor from a product type

Add

```
template <class PT> E
 EXPLICIT constexpr pair(PT&& u);
```

Let where `Ui` is `product_type::element<i, decay_t<PT>>::type` .

*Effects:* For all `i` , the constructor initializes the `i` th element of `*this` with `std::forward(product_type::get(u))`.

*Remarks:* This function shall not participate in overload resolution unless `PT` is a product type with 2 elements and `is_constructible<Ti, Ui&&>::value` is true for all `i` . The constructor is explicit if and only if `is_convertible<Ui&&, Ti>::value` is false for at least one `i` .

## Assignment from a product type

```
template <class PT>
 pair& operator=(PT&& u);
```

Let `Ui` is `product_type::element<i, decay_t<PT>>::type` .

*Effects:* For all `i` in `0..1`, assigns `std::forward<Ui>(product_type::get<i>(u))` to `product_type::get<i>(*this)`

*Returns:* `*this`

*Remarks:* This function shall not participate in overload resolution unless `PT` is a product type with 2 elements and `is_assignable<Ti&, const Ui&>::value` is true for all `i` .

## Implementability

This is not just a library proposal as the behavior depends on Structured binding [P0326R0](#). There is no implementation as of the date of the whole proposal paper, however there is an implementation for the part that doesn't depend on the core language [PT\\_impl](#) emulating the cases 1 and 2.

# Open Questions

The authors would like to have an answer to the following points if there is any interest at all in this proposal:

- Do we want the `std::product_type::size` / `std::product_type::get` functions?
- Do we want the `std::product_type::size` / `std::product_type::element` traits?
- Do we want to adapt `std::tuple_cat`
- Do we want to adapt `std::apply`
- Do we want the new constructors for `std::pair` and `std::tuple`
- Do we want the `pt_size` / `pt_get` operators in a future proposal?

## Future work

**Add `bitfield_ref` class and allow product type function access for bitfield members**

---

**Add other algorithms on Product Types**

---

### `for_each`

```
for_each : PT(T) x (T->void) -> void
```

### `front`

```
front: PT(T) -> T
```

### `back`

```
back: PT(T) -> T
```

### `is_empty`

```
is_empty : PT(T) -> bool
```

### `lexicographical_compare`

```
lexicographical_compare: PT(T) x PT(T) x (T×T->Bool) -> bool
```

The following algorithms needs a `make<TC>(args...)` factory [P0338R0](#).

If the first product type argument is `TypeConstructible` from the `CTypes` then return an instance of it, else

construct a `std::tuple` .

**cat**

```
cat: TCPT(T) ... -> TCPT(T)
```

**drop\_front**

```
drop_front: TCPT(T) -> TCPT(T)
```

**drop\_back**

```
drop_back: TCPT(T) -> TCPT(T)
```

**group**

```
TCPT(T) -> TCPT(TCPT(T))
```

**insert**

```
insert: TCPT(T) x unsigned x T -> TCPT(T)
```

**transform**

```
transform: TCPT(T) x F -> TCPT(T)
```

...

## Acknowledgments

*Thanks to Jens Maurer, Matthew Woehlke and Tony Van Eerd for their comments in private discussion about structured binding and product types.*

*Thanks to all those that have commented the idea of a tuple-like generation on the std-proposals ML better helping to identify the constraints, in particular to J. "Nicol Bolas" McKesson, Matthew Woehlke and Tim "T.C." Song.*

*Thanks to David Sankel for revising the last version.*

## References

- [Boost.Fusion](http://www.boost.org/doc/libs/1600/libs/fusion/doc/html/index.html) Boost.Fusion 2.2 library

<http://www.boost.org/doc/libs/1600/libs/fusion/doc/html/index.html>

- [Boost.Hana](#) Boost.Hana library

<http://boostorg.github.io/hana/index.html>

- [N4381](#) Suggested Design for Customization Points

<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/n4381.html>

- [N4387](#) Improving pair and tuple, revision 3

<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/n4387.html>

- [N4475](#) Default comparisons (R2)

<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/n4475.pdf>

- [N4527](#) Working Draft, Standard for Programming Language C++

<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/n4527.pdf>

- [N4532](#) Proposed wording for default comparisons

<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/n4532.html>

- [P0017R1](#) Extension to aggregate initialization

<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/p0017r1.html>

- [P0091R1](#) Template argument deduction for class templates (Rev. 4)

<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0091r1.html>

- [P0095R1](#) Pattern Matching and Language Variants

<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0095r1.pdf>

- [P0144R2](#) Structured Bindings

<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0144r2.pdf>

- [P0197R0](#) Default Tuple-like Access

<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/p0197r0.pdf>

- [P0217R1](#) Proposed wording for structured bindings

<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0217r1.html>

- [P0311R0](#) A Unified Vision for Manipulating Tuple-like Objects

<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0311r0.html>

- [\*P0326R0\*](#) *Structured binding: alternative design for customization points*

<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0326r0.pdf>

- [\*P0338R0\*](#) *C++ generic factories*

<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0338r0.pdf>

- [\*PT\\_impl\*](#) *Product types*

[https://github.com/viboes/std-make/blob/master/doc/proposal/reflection/product\\_type.cpp](https://github.com/viboes/std-make/blob/master/doc/proposal/reflection/product_type.cpp)